

Building a Kubernetes Cluster on AWS with Kops (Kubernetes Operations)

— Hands-On DevOps Practice

I recently worked through the end-to-end process of provisioning a **Kubernetes cluster on AWS using Kops**, reinforcing my practical understanding of Kubernetes infrastructure, AWS services, IAM, networking, and cluster administration.

Here's a high-level walkthrough of the implementation:

1. Prepare the AWS Environment

I created an **EC2 instance** to serve as the administration server for deploying and managing the Kubernetes cluster.

I also configured:

IAM User & Permissions

- Created an IAM user for Kops
- Assigned the required AWS permissions
- Generated an **Access Key and Secret Access Key**
- Configured AWS CLI credentials on the server

SSH Authentication

After connecting to the EC2 instance through SSH, I generated an SSH key pair:

```
ssh-keygen
```

2. Install AWS CLI

Installed and configured the **AWS CLI** so the server could interact with AWS resources.

```
aws configure
```

3. Install Kops

Downloaded the latest Kops binary:

```
curl -Lo kops https://github.com/kubernetes/kops/releases/download/$(curl -s https://api.github.com/repos/kubernetes/kops/releases/latest | grep tag_name | cut -d '"' -f 4)/kops-linux-amd64
```

Made it executable:

```
chmod +x kops
```

```
sudo mv kops /usr/local/bin/kops
```

Verified the installation:

```
kops version
```

4. Install kubectl

Installed **kubectl** to interact with the Kubernetes cluster:

```
curl -LO https://dl.k8s.io/release/$(curl -L -s  
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl
```

Then installed it into the system path:

```
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

Verify:

```
kubectl version --client
```

5. Configure S3 & Route 53

Back in the AWS environment, I configured:

Amazon S3 — to store the Kops cluster state.

Amazon Route 53 — created a hosted zone to provide DNS management for the Kubernetes cluster.

6. Create the Kubernetes Cluster

With the infrastructure ready, I created the cluster:

```
kops create cluster \  
--name=<YOUR_HOSTED_NAME> \  
--state=s3://<YOUR_S3_BUCKET> \  
--zones=us-east-1a,us-east-1b \  
--node-count=2 \  
--node-size=t3.small \  
--control-plane-size=t3.medium \  
--dns-zone=<YOUR_HOSTED_NAME> \  
--node-volume-size=12 \  
--control-plane-volume-size=12 \  
--ssh-public-key ~/.ssh/id_ed25519.pub
```

Then applied the cluster configuration:

```
kops update cluster \  

```

```
--name=<YOUR_HOSTED_NAME> \  
--state=s3://<YOUR_S3_BUCKET> \  
--yes \  
--admin
```

7. Validate the Cluster

After allowing approximately **15 minutes** for AWS resources and Kubernetes components to initialize, I validated the cluster:

```
kops validate cluster \  
--name=<YOUR_HOSTED_NAME> \  
--state=s3://<YOUR_S3_BUCKET>
```

Finally, I confirmed the Kubernetes nodes:

```
kubectl get nodes
```

Cluster up and running!

```
~# kops validate cluster --name=kopstest.adekunleadediran.com --state=s3://kopstestbucket  
Validating cluster kopstest.adekunleadediran.com  
  
INSTANCE GROUPS  
NAME                ROLE           MACHINETYPE  MIN  MAX  SUBNETS  
control-plane-us-east-1a  ControlPlane  t3.medium    1    1    us-east-1a  
nodes-us-east-1a        Node          t3.small     1    1    us-east-1a  
nodes-us-east-1b        Node          t3.small     1    1    us-east-1b  
  
NODE STATUS  
NAME                ROLE           READY  
i-00201a8c2f2817016  node          True  
i-0976466d8bd70434e  node          True  
i-0ab51bb4c140d0586  control-plane True  
  
Your cluster kopstest.adekunleadediran.com is ready
```

```
~# kubectl get nodes  
NAME                STATUS    ROLES    AGE   VERSION  
i-00201a8c2f2817016  Ready    node     24m   v1.36.3  
i-0976466d8bd70434e  Ready    node     24m   v1.36.3  
i-0ab51bb4c140d0586  Ready    control-plane 26m   v1.36.3
```

Name	Instance ID	Instance state	Instance type	Status check
kops	i-0ada8f27c4bf23a7b	Running	t3.micro	3/3 checks pas
control-plane-us-east-1a.masters.kopstest.adeku...	i-0ab51bb4c140d0586	Running	t3.medium	3/3 checks pas
nodes-us-east-1a.kopstest.adekunleadediran.com	i-00201a8c2f2817016	Running	t3.small	3/3 checks pas
nodes-us-east-1b.kopstest.adekunleadediran.com	i-0976466d8bd70434e	Running	t3.small	3/3 checks pas

VPC > Your VPCs > vpc-0246a50d613106bdd

C dashboard < vpc-0246a50d613106bdd / kopstest.adekunleadediran.com Actions

er by VPC

5 Global View

tual private cloud

r VPCs

nets

te tables

rnet gateways

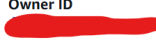
ss-only internet

eways

ier gateways

IP option sets

Details Info

VPC ID  vpc-0246a50d613106bdd	State  Available	Block Public Access  Off	DNS hostnames Enabled
DNS resolution Enabled	Tenancy default	DHCP option set dopt-094a8d371673f8be0 / kopstest.adekunleadediran.com	Main route table rtb-076b4d731600a7464
Main network ACL acl-0690478fa9c99a426	Default VPC No	IPv4 CIDR 172.20.0.0/16	IPv6 pool Amazon  Associated
IPv6 CIDR (Network border group) 2600:1f18:4aed:5e00::/56 (us-east-1)  Associated	Network Address Usage metrics Disabled	Route 53 Resolver DNS Firewall rule groups -	Owner ID 
Encryption control ID	Encryption control mode -		

This exercise provided valuable hands-on experience across several important DevOps areas:

AWS EC2 | IAM | S3 | Route 53 | AWS CLI | Linux | SSH | Kops | Kubernetes | kubectl | Cloud Infrastructure

What I particularly appreciate about Kops is how it brings together multiple infrastructure components required for a production-style Kubernetes environment while still giving you visibility and control over the underlying AWS resources.

Every hands-on deployment strengthens the connection between **cloud infrastructure, networking, security, automation, and Kubernetes administration.**

I am available for production ready collaboration, start-ups and remote jobs opportunity

#DevOps #Kubernetes #AWS #Kops #CloudComputing #AWSCloud #Infrastructure #CloudEngineering #DevOpsEngineer #Linux #Kubectl #LearningByDoing